Expert Systems with Applications 40 (2013) 3207–3219

Contents lists available at SciVerse ScienceDirect

Expert Systems with Applications

journal homepage: www.elsevier.com/locate/eswa

Agent-based game-theoretic model for collaborative web services: Decision making analysis

Babak Khosravifar [a], Jamal Bentahar [b,*], Rabeb Mizouni [c], Hadi Otrok [c], Mahsa Alishahi [b], Philippe Thiran [d]

[a] Faculty of Art and Science, McGill University, Montreal, Canada
[b] Faculty of Engineering & Computer Science, Concordia University, Montreal, Canada
[c] College of Engineering, Khalifa University of Science, Technology, & Research, Abu Dhabi, United Arab Emirates
[d] PReCISE Research Centre, University of Namur & Sirris, Brussels, Belgium

ARTICLE INFO

Keywords:
Agent computing
Web service
Decision making
Payoff

ABSTRACT

In agent and (web) service computing, collaboration takes place when distributed entities have limited knowledge and capabilities, so they cannot perform required tasks without interacting and helping each other. For instance, web services, which are loosely-coupled business applications, are called to cooperate in distributed settings for the sake of efficiency. In this context, agents that abstract and act on behalf of web services could act in cooperative groups that gather a number of agents sharing some common goals. Enabling those agent-based web services to decide about their strategies in terms of joining and acting within groups, inviting other agents to join, and leaving a group to act alone is an open issue that we address in this work. In this paper, we propose a framework where agent-based web services select strategies that maximize their outcomes. These strategies could be categorized into cooperative strategies involving other agents and strategies that highlight the single operative attitude. Although cooperation seems to bring better utility to cooperative agents, we highlight that web services in some environments obtain better outcome while they act individually (i.e., outside the group). This means that the cost of cooperation (in some particular cases) might negatively influence the outcome and obtained utility. As solution, we propose in this paper (1) an agent-based model that formalizes web services decision making where different parameters are considered; and (2) a game-theoretic framework that analyzes the web services strategies allowing them to maximize their acting performance where non-zero-sum games are being used. The paper presents theoretical results, which are also confirmed through extensive simulations.

© 2012 Elsevier Ltd. All rights reserved.

1. Introduction

A multi-agent system (MAS) is any system that is composed of multiple interacting agents playing various roles and following different acting strategies towards achieving their predefined goals. Each agent is in fact a decision maker that seeks to effectively accomplish its goals. Some of these goals are shared with other agents while others are only defined in the individual agent's desires. Because agents have limited capabilities and knowledge and can observe the environment only partially, they are called to interact with one another. The network of web services is an example of MAS where agents represent and abstract web services and act on behalf of them to make decisions regarding their cooperative attitudes (García-Sáncheza, Valencia-Garcíaa, Martínez-Béjarb, & Fernández-Breis, 2009; Lomuscio, Qu, & Solanki, 2012; Vidonia, García-Sánchezb, Gasparettoa, & Martínez-Béjarb, 2011).

A web service with an associated intelligent agent is capable of providing some services in some certain domains (service category). To do this, the agent-based web service, called web service agent, maintains some interactions with the environment and gets involved in group tasks hoping to enhance its productivity (Bentahar, Khosravifar, Serhani, & Alishahi, 2012). In this setting, a typical service consumer is an intelligent agent capable of comparing different service qualities and based on its domain knowledge, attempts to obtain a high quality service.

In this type of agent-service computing (Kun, Manwu, Hong, & Jian, 2010), each agent web service still acts individually, but it is the resulting joint action that produces the outcome (Charif & Sabouret, 2011). Collaboration is therefore a crucial aspect that improves web services's performance, robustness and scalability (Yahyaoui, 2012). The goal of cooperation is to improve the outcomes for the group as a whole while the key aim of agent-based web services is to maximize their individuals' outcome in terms of business opportunities and performance. In this context, performance is defined as the extent to which a web service agent

* Corresponding author. Tel.: +1 514 848 2424; fax: +1 514 848 3171.
E-mail address: bentahar@ciise.concordia.ca (J. Bentahar).

0957-4174/$ - see front matter © 2012 Elsevier Ltd. All rights reserved.
http://dx.doi.org/10.1016/j.eswa.2012.12.034

is successful in accomplishing its goals. This is achieved when it can effectively use its resources and capabilities to accomplish the required tasks, which allows obtaining higher outcome. Adversely, a web service agent is unsuccessful (i.e., poorly efficient) when it fails to manage its resources due to either high service demand or no demand. To this end, the web service's goal is to maximize its performance with respect to its individual capabilities.

To address web service cooperation, there have been efforts attempting to model and analyze collaborations within *communities* of web services (Khosravifar, Bentahar, Thiran, Moazin, & Guiot, 2009; Rosario, Benveniste, Haar, & Jard, 2007; Ruth & Shengru, 2007). Communities are frameworks that gather functionally-similar web service agents that share a common goal (Khosravifar, Bentahar, Moazin, & Thiran, 2010; Maamar, Subramanian, Thiran, Benslimane, & Bentahar, 2009). In the context of communities, we distinguish web services collaboration from web services composition (Yeung, 2011). By collaboration, we mean that the community aggregates web services capable of interacting with one another to manage allocated tasks, for example by allowing a web service to replace another that is incapable of executing a task. By composition, we mean extension initiated by a web service to satisfy a specific request. The web service agent invokes another web service, waits until its execution completes and uses the results of its execution, which results in a type of composite service. Similar web services providing flight booking, hotel booking, and car booking with different qualities of service, such as http://www.Orbitz.com and http://www.cheapoair.com can be gathered in a community. Another concrete example is a community of web services providing healthcare such as http://www.medcommons.net and http://www.hipaaspace.com.

In all the aforementioned frameworks of communities, the objective is to increase the overall performance where web services shape a distributed system. However, in such frameworks, strategies that web services can follow to achieve this goal are limited to a form of aggregation-based collaboration. More elaborated strategies considering when to collaborate and when to act alone are yet to be analyzed and investigated. Such strategies can help communities and individual web services achieve higher performance in using their resources. *One of the challenging issues to be addressed* is to determine if being involved in a community to cooperate with other agent web services will necessarily end up with higher outcome for an individual web service agent with some limited capabilities. In the affirmative case, identifying the best time for an individual web service agent to join a community to cooperate with others is another issue not resolved yet in the literature.

1.1. Motivations

The aim of this paper is to investigate strategies representing long-term rational behaviors agent-based web services and communities can select to increase their respective performance. We use game-theoretic principles that have been advocated to design cooperative and competitive, self-interested multi-agent systems (Pendharkar, 2012). More precisely, we present a game-theoretic model in which web services can either act alone or cooperate with other web services within a community. Each entity (single web service or community of web services) manages its reputation, market share, capacity, and performance parameters. Using our proposed framework, interactive services are capable of making decisions in uncertain and stochastic MAS environment. In fact, agents follow their own goals and therefore, their acts are not fully predictable. In such models, incentives could deviate decision-making processes that are maintained by agents from particular directions that might be socially optimal for the whole network. Therefore in this paper, a game is defined between a particular single web service and the representative of a typical community (called *master web service*). Each entity looks for better performance following strategies of *joining/leaving* a community, *accepting/refusing* a request to join a community, and *inviting* to join a community from the master. In different scenarios, we investigate the situation that maximizes players' performance.

1.2. Contributions

Overall contributions of the paper are threefold: (1) we provide a distributed network of web services and consumers where the task allocation problem is regulated by a mechanism taking reputation, market share, and performance into account; (2) we propose a game-theoretic analysis investigating the stabilized situation within which entities achieve high performance where non-zero-sum games are used; and (3) we identify thresholds allowing the master web service of a community to identify the optimal number of web services to be part of the community. We also provide experiments that show and uphold the impact of our game-theoretic analysis on the behavior of rational web services.

1.3. Paper organization

The rest of the paper is organized as follows. In Section 2, the components of the model we are considering are introduced. This includes web services, communities, service consumer agents, the feedback system, our reputation mechanism, and three key parameters: market share, capacity, and risk. Section 3 presents our game-theoretic model where the computation of the web services's and communities's performance is investigated. The scenario of the web service being outside the community and the scenario of the web service being part of the community are analyzed using non-zero-sum games. Section 4 introduces our simulation settings and results to confirm the theoretical findings. Section 5 discusses relevant related work. Finally Section 6 concludes the paper.

2. The model architecture

Before introducing our agent-based game-theoretic model, we present in this section its architecture components, which are the core of our framework. The purpose is to introduce the different concepts, which will be used through the paper.

2.1. Web service

In this paper, we consider a web service as a software application associated with a rational agent that is capable of providing a service (Kun et al., 2010) (a web service agent i is denoted by w_i). The agent represents and acts on behalf of the web service and its goal is to maximize its individual income. We define income as the total obtained fee that compensate the provided service. In short-term, the goal of a typical web service agent would be to attract the most possible service consumers and thus, obtain relatively high service fee, not necessarily appropriate for an agent with rational behavior in the long-term. Rational agents are expected to follow goals that are achievable in long-term interactions specifically in dynamic environments. For instance, an acting strategy could yield a relatively high outcome in the short-term, while the side effect of such an action is loosing a network of consumers in the future. Consequently, the active web service agent in this case requires a mechanism that guarantees its survival in the multi-agent environment with competitors, while maximizing its income. In this mechanism, there is a number of internal and

external parameters that influence the long-term performance. Internal parameters are web service agent's limited capacity for simultaneously serving consumers (C_{w_i}) and quality of provided service (Q_{w_i}). External parameters, which are influenced by public actions, are web service agent's reputation (R_{w_i}) and market share (M_{w_i}). In long-term interacting period, the internal parameters of the web service would influence its reputation and somehow its market share. This is due to the fact that overall quality of the offered service (denoted by how well the web service responds to the requests and what quality range is provided) would rate the service consumer's satisfaction regarding their future service requests.

Considering the web service agent as an interactive decision maker entity, its behavior is defined by its attitude in responding to the service requests that are continuously initiated by service consumers. For example, one agent may decide to offer its highest built-in quality (Q_{w_i}) and allocate all the required resources to accomplish a certain requested service. Another agent may decide to handle more requests by allocating relatively less resources to each request, which will result in less quality of service than the actual quality it is capable of. The first web service that provided high quality services would obtain good reputation (which yields further higher number of requests), but less income regarding service fees compared with the second web service that coped with higher number of requests but with limited quality. There is then a variation of behaviors that with a long-term perspective brings variety of income for web service agents that charge the same service fee. The key issue for the web service agent is to investigate the tradeoff between internal parameters to obtain a better income overall.

2.2. Community of web services

We consider the community (a particular community is referred to as c_j[1]) as a set of functionally-similar web services that are virtually grouped, and where members can collaborate to achieve some shared goals (Khosravifar, Bentahar, Moazin, Maamar, & Thiran, 2010; Maamar et al., 2009). The concept of community is similar to the concept of coalition in the sense that the grouping motivation and goals are similar (Chalkiadakis & Boutilier, 2012; Yu & Xu, 2012). However, the coalition is formed on the spot in order to cope with a high load of work, whereas the community is formed off-line in order to generate services that are efficiently managed, have better visibility to the customers, and are capable of handling high-loaded requests. In addition, the community structure is dynamic as members may leave or join the community at any time, but the goal of the community and its functionality are formed off-line. In general, in a coalition members are purely cooperative as they are supposed to perform a common task; while in a community, members can also be competitive as they are selfish, have individual goals, and can perform individual tasks. The community is represented by the master web service, which is in charge of the group management (e.g., inviting new members, firing bad members, and allocating tasks). In particular, the master web service uses a task allocation mechanism to select one (or a group of) web service(s) when a customer request is received. Overall communities are objectively built to increase individual web services' performance compared to the case where they act alone. However, we will show that being a member of a community is not always an indicator of making benefits as advocated in Khosravifar et al. (2009), Maamar et al. (2009). Thus, a rational interactive web service agent needs to analyze this issue in its decision making process.

[1] Through the paper, only one community is considered. We could simply use the notation c without index j. However, for compatibility reason with the notation w_j, the notation c_j is being chosen.

As mentioned earlier, the main objective of the community is to serve a large number of requests. At the macro-level perspective, a community can be seen as a web service and thus, it holds all internal (capacity, quality, and available resources) and external (reputation and market share) characteristics. The critical issue with the community would be the performance of the group as a whole. In fact, the master web service encounters a challenge to handle large number of requests, maintain scalable task allocation, and keep up the performance at an acceptable level.

2.3. Service consumer agent

Associated to service consumers, they are agents distributed in the environment, which by default look for services (either web services or communities of web services) that match their requirements. Examples of requirements are response time, service quality and service category. In our proposed framework, the role of the service consumer agent is to categorize service agents based on their reputation and continuously initiate service requests. Obviously service agents with high market share are recognized in networks associated to relatively high number of service consumers. Over continuous search, the typical service consumer agent (u_k) triggers a service request to a specific service agent and upon service response, the customer agent posts the corresponding satisfaction feedback to the feedback pool. The posted feedback considers the quality of interaction with the service agent. The provided rates influence the service reputation in the future.

We categorize consumer agents according to their feedback mechanisms into three groups: *accurate feedback mechanism* group (AFM), *risky feedback mechanism* group (RFM), and *hybrid feedback mechanism* group (HFM). In AFM, consumer agents provide satisfaction feedback by just comparing the received quality with the expected one. Thus, a service agent could estimate the received satisfaction feedback with some certain accuracy. In RFM,consumer agents might consider some other factors while posting the satisfaction feedback, which makes it harder for the service agent to estimate this value. In these cases, service agents learn the consumer agents' attitude in posting satisfaction feedback over time. In HFM, consumer agents' attitude in posting satisfaction feedback is a combination of accuracy in expected quality comparison with some other external factors, which to some extent makes it easier for the web service agents to estimate the satisfaction feedback. We use these groups in our experimental results comparisons (Section 4). For simplicity reasons and without loosing the generality of the addressing problems, we generally assume that service consumer agents are rational users that only look for the best possible service and follow the selection algorithm presented in Fig. 1. In this algorithm, consumer agent u_k refers to the list of service agents as the set *SP* (service providers). This set is initialized to include the known service providers in u_k's subnetwork and updated by the recent joined/left service providers. Therefore, at any run time of this algorithm, *SP* represents the set of service agents that are available for u_k. This set is sorted with respect to information kept in the response history set *RH*, so that u_k can select the best options from the set *SP*. Any details regarding the way u_k can categorize the service agents could be implemented in the function *Sort*(*SP*,*RH*). In fact, *Sort*(*SP*,*RH*) represents u_k's attitude in selecting the best options. This set might be sorted with respect to response time, service quality, service fee, or an aggregation that the agent can define. The consumer agent simply picks service agents from *SP* and initiates the request. The corresponding feedback are provided even if the required service is not obtained (in case *sp* did not respond). This process is stopped once u_k obtains the required service (*service obtained* = 1). Once the service is obtained, the quality is analyzed and the corresponding feedback is posted. If no service agent has responded, then u_k

Algorithm1 Service Selection for Consumer Agent u_k

Require: Obtain service
1: **if** *first service request* **then**
2: Initialize SP
3: **else**
4: Update SP
5: Sort(SP< RH)
6: **while**((*service obtained*==0) or (*there is* $sp \in SP$)) **do**
7: Pick sp from SP ;*remove from the list*
8: Initiate Request(sp)
9: **if** *service*=TRUE **then**
10: *service obtained*=1
11: Evaluate(*service*)
12: Feedback(sp)
13: **end if**
14: **else**
15: Feedback(sp)
16: **end while**

Fig. 1. Service consumer agent's decision making algorithm that selects the best possible web service agent to initiate the service request.

remains with no service and continues to search in the next function run.

2.4. Feedback system

In our MAS design for web services, we develop a feedback pool where consumer agents post their satisfaction ratings regarding their past experiences with service agents. In this system, we assume that every consumer agent provides a post-interaction feedback. Posted feedback are accumulated to compute and analyze service agents' reputation.

The incentive for posting feedback has been addressed in some frameworks such as Jurca, Faltings, and Binder (2007), Khosravifar et al. (2009) and it could be the reimbursement to the consumer agent of a percentage of its initiated service request fee after positing the feedback. To avoid misleading feedback, the authors in Jurca et al. (2007) study the accuracy of the posted feedback and propose to make the reimbursements based on the accuracy of the feedback provided.

2.5. Reputation mechanism

In terms of value, reputation of a service agent (R_{w_i} for the web service w_i and R_{c_j} for the community c_j) is a value between 0 and 1 computed from the feedback pool. Web services and communities as rational agents aim at increasing this value, which imposes positive impact on their outcomes. However, increasing reputation brings more requests which might impose negative impact if the requests are not systematically handled. There are a number of frameworks that address the reputation via feedback aggregation, for example (Bentahar et al., 2012; Khosravifar et al., 2010; Maximilien, 2005). For instance, in Bentahar et al. (2012), reputation is computed considering quality, time discount factor, and market share. Among the existing frameworks we use a conventional mechanism with three values: (i) +1 for satisfied experience; (ii) 0 for no response to the request; and (iii) −1 for unsatisfied experience. This mechanism adds the positive values of all provided feedback for the particular service agent and divides the result by the total number of feedback. In Eq. (1), PF_{w_i} and NF_{w_i} respectively denote the number of positive and negative feedback posted for web service w_i. NR_{w_i} denotes the number of no responded requests associated to the same web service. In Eq. (1), the reputation of the community c_j is computed as the average of the reputation of all the members ($|c_j|$ is the cardinality of the set c_j).

$$R_{w_i} = \frac{PF_{w_i}}{PF_{w_i} + NF_{w_i} + NR_{w_i}} \quad R_{c_j} = \frac{\sum_{w_i \in c_j} R_{w_i}}{|c_j|} \tag{1}$$

The parameters PF_{w_i}, NF_{w_i}, and NR_{w_i} are updated on a regular basis, daily for instance. In fact, this basis depends on the MAS design and how interactive the transaction system is. Therefore, upon request, the updated reputation value is provided. Similar to the individual web services, the community of web services also holds updated reputation value (as an average of reputation of all the involved web services in he community).

Within a community, the master web service would decide how to cope with the service request. This means that the consumer agent cannot select the specific web service in the community. However, upon task allocation, the consumer agent provides the post-interaction feedback regarding the corresponding web service(s) that provided the service. In this case, active web services in the community still update their individual reputation value and influence the mean reputation value of the community as a whole.

2.6. Market share

In terms of value, the market share associated with a service agent is in the interval [0, 1]. This value reflects the percentage of the market served by a specific web service among other web services. The value is computed in Eq. (2), where Req_{w_i} denotes the requests sent to the web service w_i and $TReq$ represents the total requests filed. Ideally, by aggregation of web services, the market share assigned to the group increases considering the coverage of the community over diverse services.

$$M_{w_i} = \frac{Req_{w_i}}{TReq} \quad M_{c_j} = \sum_{w_i \in c_j} M_{w_i} \tag{2}$$

2.7. Capacity and risk analysis

The capacity parameter associated to the service providers (C_{w_i} for web service w_i and C_{c_j} for the community c_j) denotes the maximum number of simultaneous services that the service agent can handle. Obviously, grouping web services together increases the group's ability to handle more simultaneous service requests. In Eq. (3), the capacity of the community c_j is computed by the sum of capacity values of all the allocated web services in the community.

$$C_{c_j} = \sum_{w_i \in c_j} C_{w_i} \tag{3}$$

As mentioned previously, web services and communities have different strategies in allocating the available resources to the service requests. This allocation reflects in fact the extent to which the service provider can put effort on serving the requesting consumer agents. It is reflected by a percentage of the maximum quality of service for the typical web service $w_i(Q_{w_i})$. If 100% of the required resources are allocated, the offered service would be at quality of Q_{w_i} as long as the web service w_i is in fact capable of serving at this quality. But if for example 80% of the required resources are allocated, the offered service would be at quality of $0.8Q_i$. This would deduce the offered quality and therefore might negatively influence the satisfaction feedback, but keep more available resources, enabling the web service to serve more requests and therefore earn more service fee. Hence, once web service agents receive service fee upon serving the consumers and since agents might be selfish, some would desire to increase the accepting requests to serve to

obtain more income. However, due to the fixed available resources, these agents are taking the risk of lowering their offered quality of service. In the following, we highlight relevant assumptions by which agents may take that risk without affecting their initial goal of maximizing the obtained utility.

(1) **High reputation**: When a web service already holds a relatively high reputation, it may receive a high number of service requests that could exceeds its capacity. Decreasing its reputation is not of concern for such web service as long as the decreased number of requests still exceeds the web service's capacity. Consequently, a web service with already high reputation is more amenable to take the risk of decreasing the offered service quality compared to the one that is looking to increase its reputation to receive more requests.
(2) **High market share**: A web service that holds a relatively high market portion is more guaranteed to receive continuous requests compared to the one that has a small portion. In general, this idea does not last long because the negative feedback regarding bad service quality would propagate in the network and influence the market share. But still a malicious web service with high market share might consider taking the risk for a limited time interval.
(3) **High gain**: When the gained amount of service fee is relatively high, a web service might take the risk to variate its service quality to try the chances of gaining more income. Obviously, recently activated web services are aimed at increasing their reputation and market share, so such a risky behavior is not expected from them. Eq. (4) computes the total extra gained amount associated to web service w_i or the lack of gain for a period of D days. In this equation, g_{d,w_i} (which could be positive or negative) represents the extra amount that web service w_i gained by accepting more requests than its associated capacity (if $Rq_{d,w_i} - C_{w_i}$ is positive), or the lack to be gained (if $Rq_{d,w_i} - C_{w_i}$ is negative), where Rq_{d,w_i} is the serving requests at day d and β is the fixed service fee over the network. In fact, the parameter β denotes the service fee that we associate to any service that is being provided by a web service and is the amount paid to the web service by the user. We consider using β to be able to convert all expected payoffs to money, so that we can quantify and analyze the web service's budget.

$$G_{w_i} = \sum_{d=1}^{D} g_{d,w_i} \qquad g_{d,w_i} = (Rq_{d,w_i} - C_{w_i}) \times \beta \tag{4}$$

The increase and decrease in one's service requests influence its overall performance, and besides the performance, the long-term goal of the service agent w_i is to maximize its gained value G_{w_i}. This long-term goal is achievable when the rational agent always keeps his performance high. The active agent is successful in long-term interacting environments when its performance is always acceptable.

3. The game theoretic model

In this section, we analyze the characteristics of the proposed model and highly emphasize the details of the performance parameter.

3.1. Performance computation

In general, service agents tend to maximize their performance and achieve their long-term goals. We propose a heuristic (see Eq. (5)) for computing the performance E_x as a function f of reputation R_x, market share M_x, obtained service requests Rq_x, and capacity C_x where $x \in \{w_i, c_j\}$.

$$E_x = f(R_x, M_x, Rq_x, C_x) \tag{5}$$

The function f should satisfy the following properties.

Property 1. *f is continuous.*

This property says that at each moment, the performance of a web service or a community can be evaluated with respect to the current attributes.

Property 2. *f is strictly increasing in R_X and M_X.*

This property says that the performance of the web service increases if it holds high reputation and market share in the system. Consequently, web services and communities will have incentives to increase their reputation and market share to get their overall performance increased.

Property 3. *f is monotonically decreasing in $Rq_X - C_X$.*

This property says that the performance of a web service or a community decreases if it fails to make a good balance between its capacity and the requests it should handle. Consequently, web services and communities will have incentive to analyze their capacities and manage to have acceptable market share. The idea is that the more service provider entity succeeds in making balance between its capacity and received requests, the higher the performance would be.

Eq. (6) gives a possible definition of f.

$$E_x = \frac{R_x \times M_x}{|Rq_x - C_x| + 1} \tag{6}$$

Theorem 1. *The function f satisfies* Properties 1–3.

Proof. Satisfaction of Property 1 is straightforward as all the parameters are defined at each moment in time, so the function is continuous. Property 2 can be proved by computing the partial derivatives $\frac{\partial f}{\partial R_x}$ and $\frac{\partial f}{\partial M_x}$, which are clearly positive. Property 3 can be proved by considering $|Rq_x - C_x|$ as a variable, say v and compute the partial derivative $\frac{\partial f}{\partial v}$, which is manifestly negative $\left(\frac{-R_x \times M_x}{(|Rq_x - C_x|+1)^2}\right)$, so we are done. □

In order to hold high performance, web service agents consider the following strategies: joining a community, accepting or inviting to join a community, leaving the community, etc. Prior to acting, web service agents consider how the chosen strategy might affect their performance. Obviously, rational agents do not attempt to decrease their performance; however, they might undergo some risky actions which result in decreasing the performance in the short-term while increasing it in the long-term. This is an attribute that categorizes service agents. We refer to it as the risk factor S_X. This factor indicates how flexible the service agent entity is in loosing its performance. For example, if the risk factor associated with w_i is 20% ($S_{w_i} = 0.20$), then the web service w_i would consider any situation in its strategy analysis where estimated performance is more than 80% of its current performance. $\overline{E_{w_i}}$ is defined as the estimated performance of the web service w_i after taking any strategy for updating its status ($\overline{E_{c_j}}$ corresponds to the community c_j). To this end, the web service w_i would discard all the strategies (and

choices of updating the current status) that result in an estimated performance less than $(1 - S_{w_i})E_{w_i}$.

Using the risk factor adds flexibility to the web service and communities in choosing strategies. In the rest of this section, we discuss two different cases where the web service is outside and inside the community. In each case, we analyze the best strategies that culminate in a better performance level for both the web service and community.

3.2. Web service out of community

In this scenario, web service w_i acts individually and holds the performance level of E_{w_i}. Considering its chances of increasing its self-performance, w_i is facing the community c_j with different strategies that would end in either the single web service w_i joins the community c_j or not. This action could be initiated or ceased either by the web service or the community representative (the master). Doing so, there are four different possible actions: (a) w_i attempts to join c_j and the attempt is accepted; (b) w_i attempts to join, but c_j refuses the join request; (c) c_j invites the web service w_i but w_i refuses the invitation; and (d) there is neither invitation from c_j nor join request from w_i. From the outcome perspective, the cases of "w_i attempts to join and c_j accepts" and "c_j invites and w_i accepts" are similar. However, refusal from any party would lead to different estimating performances and this is why we consider them as two distinguished actions. In the following, we compute the estimated performance of each entity with respect to the taken action.

Case (**a**) Joining the community will have impact on the parameters of the web service. In fact, the web service w_i that takes the risk of join (S_{w_i}) would update its reputation, market share and capacity parameters respectively as shown in Eqs. (7) and (8).

$$\overline{R_{w_i}} = \frac{|c_j| \times R_{c_j} + R_{w_i}}{|c_j| + 1} \tag{7}$$

$$\overline{C_{w_i}} = C_{w_i} \quad \overline{M_{w_i}} = \frac{M_{c_j} + M_{w_i}}{|c_j| + 1} \tag{8}$$

In this case, (1) the reputation of a web service would be updated to the average of the community reputation. To this end, each registered web service in the community holds its individual reputation, but broadcasts the public reputation of the community; and (2) the capacity of the web service stays unchanged since it is a fixed attribute. The community accumulates the joined web service's capacity and market share. In addition, the joined web service is going to obtain a share of the total market share from the community. The corresponding attribute updates regarding the community c_j are formulated in Eqs. (9) and (10).

$$\overline{R_{c_j}} = \overline{R_{w_i}} \tag{9}$$

$$\overline{C_{c_j}} = C_{c_j} + C_{w_i} \quad \overline{M_{c_j}} = M_{c_j} + M_{w_i} \tag{10}$$

In this case, both entities consider the estimated parameters and compute their new performance values (see Eq. (6)). The case would take place when the following inequalities hold:

$$\overline{E_{w_i}} \nless (1 - S_{w_i})E_{w_i} \quad \overline{E_{c_j}} \nless (1 - S_{c_j})E_{c_j}$$

Case (**b**) w_i requests joining the community, but the community does not accept the request. The difference between the cases (a) and (b) is that in case (a) the join takes place, which brings actual updated performance for both entities. However, in case (b) the join does not take place, which keeps the analysis at the estimation level. The corresponding estimated performances are characterized by the following inequalities:

$$\overline{E_{w_i}} \nless (1 - S_{w_i})E_{w_i} \quad \overline{E_{c_j}} \ngtr (1 - S_{c_j})E_{c_j}$$

Case (**c**) c_j invites the web service w_i to join the community but the web service does not accept the request. Recall that any rational entity decides based on the expected performance level compared with the weighted (by risk factor) current performance level. The corresponding estimated performances are characterized by the following inequalities:

$$\overline{E_{w_i}} \ngtr (1 - S_{w_i})E_{w_i} \quad \overline{E_{c_j}} \nless (1 - S_{c_j})E_{c_j}$$

Case (**d**) Both entities (the community c_j and web service w_i) are not encouraged to attempt joining and therefore, the join does not take place. In this case, we have:

$$\overline{E_{w_i}} \ngtr (1 - S_{w_i})E_{w_i} \quad \overline{E_{c_j}} \ngtr (1 - S_{c_j})E_{c_j}$$

3.3. The game set up for single web service

Upon the discussed cases, we develop a game-theoretic model consisting of the web service w_i as player 1 and community c_j as player 2 playing a non-zero-sum game. The player 1 follows the strategy profile of (join/not join) when is initiating the game (i.e. play first), and follows the strategy profile of (accept join/refuse join) when is reacting to the opponent's move (i.e. play second). Since for our analysis it is only important whether the join takes place or not, the order of playing does not matter when calculating payoffs (represented in terms of performance). Table 1 shows the assigned payoffs for both players in different cases. As shown in the table, the values of J_{w_i,c_j} and A_{w_i,c_j} are the generalized form of "join/accept" or "invite/accept join" cases. These values are actual differences in performance values after the join (E'_{w_i} and E'_{c_j}) with their current performance levels. The obtained payoffs could be either positive or negative. The negative payoff denotes the wrong decision the entity regrets. The payoffs obtained in the other cases are all upon estimations.

The developed game is only a one-stage game between a typical web service and a typical community. The game could be set up between any other two entities and is repeated over time when entities are active in the network. Moreover, rational entities consider the information obtained in one game in their further strategy analysis. We formalize the results we obtain from the set up game between these entities in the following.

Proposition 1. *In one-stage game, there is no pure strategy Nash equilibrium.*

Proof. In the set up one-stage game, the payoff of web services regarding accepted join request (J_{w_i,c_j}) could be either more or less than that of refusing the invitation (as it refers to the actual performance evaluation). This is also the case for the master of the com-

Table 1
Payoff regarding 2 players when web service is outside the community.

Web service (Player 1)	Community (Player 2): Accept/Invite Join	Community (Player 2): Refuse/Not Invite Join
Join/Accept Invitation	J_{w_i,c_j}, A_{w_i,c_j}	$\mathcal{JR}^{c_j}_{w_i}, \mathcal{JR}^{w_i}_{c_j}$
Stay/Refuse Invitation	$\mathcal{SI}^{c_j}_{w_i}, \mathcal{SI}^{w_i}_{c_j}$	0 , 0

$$J_{w_i,c_j} = E'_{w_i} - E_{w_i}$$
$$A_{w_i,c_j} = E'_{c_j} - E_{c_j}$$
$$\mathcal{JR}^{c_j}_{w_i} = E_{w_i} - \overline{E_{w_i}}$$
$$\mathcal{JR}^{w_i}_{c_j} = E_{c_j} - \overline{E_{c_j}}$$
$$\mathcal{SI}^{c_j}_{w_i} = E_{w_i} - \overline{E_{w_i}}$$
$$\mathcal{SI}^{w_i}_{c_j} = E_{c_j} - \overline{E_{c_j}}$$

munity. Consequently, there is no dominant strategy for any player. Therefore, no pure strategy Nash equilibrium can be found. □

As a consequence of this proposition, there is no stable situation rational entities can try to achieve by playing the game. Both players should then consider the risk parameter in their strategy selections. To this end, we define web service and community's mixed strategy probabilities respectively as $w_i(S_{w_i}, 1 - S_{w_i})$ and $c_j(S_{c_j}, 1 - S_{c_j})$. Thus, we compute web services expected payoff α_{w_i} of join (or accept to join) considering the mixed strategy profile of the community in Eq. (11). Eq. (12) computes the related value regarding the refusal of join.

$$\alpha_{w_i}(join, c_j(S_{c_j}, 1 - S_{c_j})) = S_{c_j}(J_{w_i,c_j}) + (1 - S_{c_j})\left(JR_{w_i}^{c_j}\right) \tag{11}$$

$$\alpha_{w_i}(stay, c_j(S_{c_j}, 1 - S_{c_j})) = S_{c_j}\left(SI_{w_i}^{c_j}\right) + (1 - S_{c_j})(0) = S_{c_j}\left(SI_{w_i}^{c_j}\right) \tag{12}$$

The web service aims at maximizing its payoff. Therefore, for all adopted strategies, we need to consider the best response (to the other player) and discard the others. For instance, if the web service obtains a higher expected payoff with the joining strategy, it would change its probability profile to (1,0), so the join would be the dominant strategy.

Since each player in each stage game chooses between only two strategies, and since any of these strategies could be the best response in a particular situation, we analyze the case where the expected payoffs are equal. By so doing, we can compute a threshold (μ_{w_i}), which is used to identify which strategy is dominant. The threshold μ_{w_i} is used by the master web service to control the expected payoff of the web service in the sense that the web service adopts the master's desirable strategy as dominant. Thus, the master would pay the least possible cost to obtain its desirable control on the web services. This eventually would lead to a control mechanism of the master web service over the cardinality of the community set. The threshold is computed in Eq. (13).

$$\begin{aligned}
\alpha_{w_i}(join, c_j(S_{c_j}, 1 - S_{c_j})) &= \alpha_{w_i}(stay, c_j(S_{c_j}, 1 - S_{c_j})) \\
&\Rightarrow S_{c_j}(J_{w_i,c_j}) + (1 - S_{c_j})\left(JR_{w_i}^{c_j}\right) \\
&= S_{c_j}\left(SI_{w_i}^{c_j}\right) \\
&\Rightarrow S_{c_j}\left(E'_{w_i} - E_{w_i}\right) + (1 - S_{c_j})(E_{w_i} - \overline{E_{w_i}}) \\
&= S_{c_j}(E_{w_i} - \overline{E_{w_i}}) \Rightarrow \mu_{w_i} \\
&= \frac{(S_{c_j})\left(E'_{w_i}\right) + E_{w_i} - 3(S_{c_j})(E_{w_i})}{1 - 2S_{c_j}}
\end{aligned} \tag{13}$$

The threshold μ_{w_i} obtained in Eq. (13) should be compared against the estimated performance $\overline{E_{w_i}}$, which could be changed by the master c_j. So if the expected performance of join is computed to be more than μ_{w_i}, the web service w_i would adopt the join or accept the invitation to join strategy. We have then the following result.

Proposition 2. *In the mixed strategy one-stage game, there is a threshold μ_{w_i} such that if $\overline{E_{w_i}} > \mu_{w_i}$, joining the community would be the goal of the web service. Otherwise, the web service w_i would not join the community.*

Corollary 1. *If the master web service considers the expected performance value computed by the web service and provides (broadcasts) a reputation value that let $\overline{E_{w_i}}$ exceeds μ_{w_i}, the master can control the adopting strategy of the web service.*

3.4. Web service in the community

In the previous section, we analyzed the cases where the web service w_i was acting alone outside the community c_j. We set up a non-zero-sum game and analyzed the payoffs regarding different adopting strategies to either join or not the community. In this part, we analyze the scenario where the web service w_i is already acting in collaboration with other web services inside the community c_j. In this case, the web service chooses its actions from strategy profile of "leave/accept to leave" or "stay/refuse to leave" (we assume that any action that ends up in changing the status of the web service is being made upon agreements between the web service and the master of the community). The community c_j also refers to the strategy profile of "accept of leave/fire" or "refuse the leave/not fire". Doing so, there are four different cases: (a) w_i attempts to leave the community c_j and the attempt is accepted; (b) w_i attempts to leave, but c_j refuses the request; (c) c_j encourages the web service w_i to leave, but w_i refuses the leave request; and (d) there is neither requesting to leave from c_j nor leaving request from w_i. Similar to the case where the web service was outside the community, we analyze the cases with their parameter updates. We include more details regarding the "leave" concept because in general, a web service is known as an independent rational entity and can leave the community at any time. This does not contradict the fact that the community can influence a web service's outcome such that the rational web service has no choice other than leave, which we consider as a firing process. When the community wants a web service to leave the group, there are still chances that the web service refuses to leave and continues operating in the community, which reflects web service's internal decision making on obtaining higher outcome while being active inside the community. To this end, we need to consider in the following different scenarios where the web service and the master of the community encounter various situations and obtain different ranges of outcomes.

Case (**a**) The web service w_i that takes the risk of leave (S_{w_i}) would update its reputation, market share and capacity parameters respectively in Eq. (14).

$$\overline{R_{w_i}} = R_{c_j} - \epsilon_R \quad \overline{C_{w_i}} = C_{w_i} \quad \overline{M_{w_i}} = \frac{M_{c_j}}{|c_j|} - \epsilon_M \tag{14}$$

The reputation of the web service w_i would be lightly less than the reputation of the community, which is calculated as the average reputation of all the members. The small difference (*i.e.*, ϵ_R) represents the contribution of the community to the web service's reputation (the community provides a cooperative setting). Generally, $\epsilon_R \ll R_{c_j}$, and is a value determined by the master. Notice that the web service could get back its individual reputation that it holds. However, such decision may encourage misbehavior where after leaving the community, the web service will exhibit an erroneous reputation to get more requests. For the community, its reputation is recalculated without considering w_i. Since the capacity is a fixed attribute,the capacity of the web service w_i stays unchanged. However, the capacity of the community is recalculated to deduce the capacity of the leaving web service. A similar analysis of the reputation can be applied for the market share where $\epsilon_M \ll M_{c_j}$ is the community's contribution to the overall market share (the community provides better visibility to the members). The corresponding attribute updates regarding the community c_j are formulated in Eq. (15).

$$\overline{R_{c_j}} = \frac{|c_j|(R_{c_j}) - R_{w_i}}{|c_j| - 1} \quad \overline{C_{c_j}} = C_{c_j} - C_{w_i} \quad \overline{M_{c_j}} = M_{c_j} - M_{w_i} \tag{15}$$

In this case, both entities consider the estimated parameters and compute their new performance values (see Eq. (6)). The case would take place when the following inequalities hold:

$\overline{E_{w_i}} \nleq (1 - S_{w_i})E_{w_i} \quad \overline{E_{c_j}} \nleq (1 - S_{c_j})E_{c_j}$

Case (**b**) In this case, w_i attempts to leave, but the community does not accept the leaving request. The difference between the cases (a) and (b) is the same as explained in the previous section. We have then the following inequalities:

$\overline{E_{w_i}} \nleq (1 - S_{w_i})E_{w_i} \quad \overline{E_{c_j}} \ngeq (1 - S_{c_j})E_{c_j}$

Case (**c**) This case is similar to the case (b) except the fact that the refusal is caused by the web service:

$\overline{E_{w_i}} \ngeq (1 - S_{w_i})E_{w_i} \quad \overline{E_{c_j}} \nleq (1 - S_{c_j})E_{c_j}$

Case (**d**) In this case, both entities are not encouraged to attempt leaving:

$\overline{E_{w_i}} \ngeq (1 - S_{w_i})E_{w_i} \quad \overline{E_{c_j}} \ngeq (1 - S_{c_j})E_{c_j}$

3.5. The game set up for the joined web service

In this section, we also develop the game-theoretic analysis consisting of the web service w_i as player 1 and community c_j as player 2 playing a non-zero-sum game. The player 1 follows the strategy profile of (leave/not leave) when is the initiator and follows the strategy profile of (accept fire/refuse fire) otherwise. Table 2 shows the assigned payoffs for both players in different cases. As shown in the table, the values of $L_{w_i.c_j}$ and $F_{w_i.c_j}$ are the generalized form of "leave/accept of leave" or "fire/accept of leave" cases. These values are actual differences in performance values after the join (E'_{w_i} and E'_{c_j}). The obtained payoffs could be either positive or negative. We formalize the results we obtain from the set up game between these entities in the following.

Proposition 3. *In one-stage game, there is no pure strategy Nash equilibrium.*

Proof. The proof is similar to the one given for Proposition 1. □

Referring to the obtained payoffs shown in Table 2, we would have the same best response analysis that we did in the case for the single web service. The obtained threshold μ_{w_i} is set the same.

Proposition 4. *In the mixed strategy one-stage game, there is a threshold μ_{w_i} such that if $\overline{E_{w_i}} > \mu_{w_i}$, leaving the community would be the goal of the web service. Otherwise, the web service w_i would stay in the community.*

Table 2
Payoff regarding 2 players when web service is inside the community.

Web service (Player 1)	Community (Player 2): Accept Leave/Fire	Community (Player 2): Not Fire/ Refuse Leave
Leave/Accept Leaving	$L_{w_i.c_j}, F_{w_i.c_j}$	$IR_{w_i}^{c_j}, IR_{c_j}^{w_i}$
Stay/Refuse Leaving	$SF_{w_i}^{c_j}, SF_{c_j}^{w_i}$	0 , 0

$$L_{w_i,c_j} = E'_{w_i} - E_{w_i}$$
$$F_{w_i,c_j} = E'_{c_j} - E_{c_j}$$
$$IR_{w_i}^{c_j} = E_{w_i} - \overline{E_{w_i}}$$
$$IR_{c_j}^{w_i} = E_{c_j} - \overline{E_{c_j}}$$
$$SF_{w_i}^{c_j} = E_{w_i} - \overline{E_{w_i}}$$
$$SF_{c_j}^{w_i} = E_{c_j} - \overline{E_{c_j}}$$

Corollary 2. *If the master web service considers the expected performance value computed by the web service and provides a market share value that let $\overline{E_{w_i}}$ exceeds μ_{w_i}, the master can control the strategy of the web service.*

The market share offered to the web service by the community could cause dissatisfaction of the joined web services. Thus, this would generate a low E_{w_i} value, which would cause the web service to leave considering its previous individual performance value.

The games we considered in this section are one-stage games. These games can be played in a repeated manner, where different strategies could be analyzed, such as tit-for-tat. Learning techniques can also be used to learn the best strategy web services and communities can play. This aspect is out of the scope of this paper and will be investigated as future work.

4. Simulation and empirical analysis

In this section, we will show how the theoretical results are confirmed by the simulations. First, we will start by discussing the simulation environment and the associated parameters. Next, simulation results will be presented and discussed.

4.1. Simulation settings

We used a realistic multi-agent simulator in a java-based platform and developed many agents with broad range of characteristics and capabilities. The implemented multi-agent environment is flexible in the sense that it hosts different types of agents with variety of functionalities, behaviors and goals. We exposed dynamism in agents' actions and therefore, we could obtain results that reflect realistic experiments. In the implemented environments, there are three types of agents: (a) service consumer agents; (b) web service agents; and (c) master web service agents that represent communities. These agents are distributed in the multi-agent network using normal distribution. Table 3 depicts the simulated entities characteristics. We categorize the web services and master agents based on the risk S_x they take in their adopting strategies. Thus, three classes of service agents that obtain different payoffs during the simulation runs are considered (S_x = 0.2, 0.5, or 0.8).

We also divide the experiments in five different environment settings (see Table 4): (a) risky feedback mechanism (RFM); (b) accurate feedback mechanism (AFM); (c) hybrid feedback mechanism (HFM); (d) goal-based agents (GBA); and (e) utility-based agents (UBA). The element that distinguishes the environment settings is the attitude of the active agents. We examine our results discussed in Section 3 in all the categorized environments. Obviously different environment settings influence our results, but overall we concluded with mechanisms that increase service agents' performance. In the first three environments, we impose different characteristics of service consumers by posting unbiased feedback. In RFM, service agents encounter risky service consumers and thus, they cannot rely on the accumulated feedback to increase self reputation. It is not the case in AFM, where service agents accumulate correct feedback according to the quality of re-

Table 3
Entities characteristics.

Entity type	Number	Risk level (S_x)
Consumer agent	1000	–
	50	0.2
Web service agent	150 50	0.5
	50	0.8
	5	0.2
Community representative	15 5	0.5
Agent	5	0.8

Table 4
Simulation environments characteristics.

Simulation environment	Running time steps
RFM	800
AFM	500
HFM	700
GBA	600
UBA	300

sponse to service requests. HFM contains both of these agents. In this system, accurate consumer agents are recognized over time and therefore, risky consumers are gradually ignored by service agents. The GBA environment contains agents that consider long-term goals and act according to their performance analysis, but agents in the UBA environment have different point of view in the sense that they are utility maximizers. They might follow strategies that bring high temporary utilities but fail to success in long-term interactions. In our implemented system, we examine the performance of all types of agents in all of these environments. But we explore further details regarding the GBA environment, which shows our proposed long-term performance analysis. The objective is to highlight the fact that GBA obtain higher long-term utilities compared with UBA. Agents in the UBA environment might take decisions that bring relatively high utility but only in short periods of time.

4.2. Simulation results and analysis

As clarified in Table 4, we run the simulated environments with variety of time steps needed to make the environment stable. In fact, the environment needs more time steps to get stable when consumer agents are risky (RFM) and hybrid (HFM). Thus, more time steps are reserved for them, so that service agents have more time to get exposed to risky and hybrid consumer agents. Moreover, GBA are given double running time steps compared to UBA because GBA are following long-term strategies. Having stable environments allows us to better observe the theocratical results discussed in the previous sections.

In Table 5, we explore some results over the simulation run in different environments. The idea is to compare the important parameters and observe how these parameters are influenced in variety of settings. All these parameters are mean values captured from the stabilized runs (obtained in Table 4) where agents do not have dramatic changes in their values.

(1) **The reputation accuracy parameter**: The reputation accuracy is the parameter that we compute as the mean of the reputation assessment in continuous runs. This value is updated at the end of each run and reflects the accurate reputation assessment for service agents. In RFM, this value is the least due to service consumer agents' attitude in providing biased feedback. Adversely, we obtained maximized reputation accuracy in the AFM environment. The hybrid HFM environment performs with lower accuracy, however HFM reputation accuracy approaches the one computed for AFM since biased service consumers do not survive in long-term interactions. The distribution of the service consumers in the GBA and UBA environments is hybrid (consumers are from HFM), so the reputation accuracy of those agents is as computed for HFM (i.e., 0.76).

(2) **The selection performance parameter**: The selection performance parameter denotes the efficiency of the service consumers in selecting the right service provider. This mean value represents the overall performance of service selection in the environment. Obviously, in the environment with accurate feedback mechanism, the reputation parameter is a correct way of categorizing service providers.

(3) **The join and leave parameters**:So far, the mentioned results are promising, but rational agents undergo continuous games and a number of parameters are taken into account while they tend to maximize their performance. To this end, the joining, leaving, and performance regarding individual web services and community of web services are parameters that we need to record in simulation runs and we cannot pre-estimate them. We show these values as a pair regarding web services and communities. Each rational fraction represents the total (of join or leave) in the denominator and the succeeded in the numerator. Regarding the joining, we observe that in HFM, there are more joining requests compared to the ones in RFM or AFM. However, the accepted ones exceed in AFM. This is more visible in GBA and UBA in the sense that agents in UBA have short-term vision, which might avoid them to request to join a community. With respect to the communities, we observe more invitation of join in HFM, but higher successful invitations in AFM. This is explained by the fact that interacting agents attempt to increase their performance in continuous runs. HFM agents are more active since there is inaccuracy in the reputation mechanism, but AFM agents are more successful in the sense that they do not encounter misleading parameters in their calculations.

(4) **The performance parameter**: Overall performance would better reflect the outcome of the continuous games involving rational agents. We observe maximum performance in the HFM environment. This value exceeds the ones in AFM and RFM. This shows that interacting agents maintain higher performance in managing their obtained service requests while they are more active compared to the case where the environment is more stabilized. This performance is more visible when we compare the GBA environment where agents are aligned with a long-term performance analysis with UBA where agents seek to maximize their short-term utilities.

In Table 6, we extract more detailed information about the GBA setting in the sense that agents follow long-term performance analysis and consumer agents are distributed in the HFM environment. In this table, we outline the average performance and rate of change of the performance, reputation, and gained value in run-

Table 5
Environmental parameters in stabilized runs.

Environment	Reputation accuracy	Selection performance	Join(Acc/Req,Acc/Inv)	Leave(Acc/Req,Acc/Fir)	Performance(WS,CWS)
RFM	0.62	0.53	(10/31,5/7)	(10/10,1/3)	(0.61,0.48)
AFM	0.96	0.82	(20/52,4/7)	(8/8,1/4)	(0.68,0.51)
HFM	0.76	0.72	(18/54,3/10)	(12/12,2/2)	(0.81,0.64)
GBA	0.76	0.77	(14/43,5/8)	(9/9,1/3)	(0.73,0.59)
UBA	0.76	0.61	(5/21,1/3)	(2/2,1/2)	(0.21,0.18)

Table 6
GBA performance in steps 300–320.

Entity	Average performance	Performance rate of change	Reputation rate of change	Gained rate of change
Single web service	0.68	0.02	−0.07	+13
Joined web service	0.79	0.11	−0.02	+7
Community	0.53	0.03	+0.12	+182

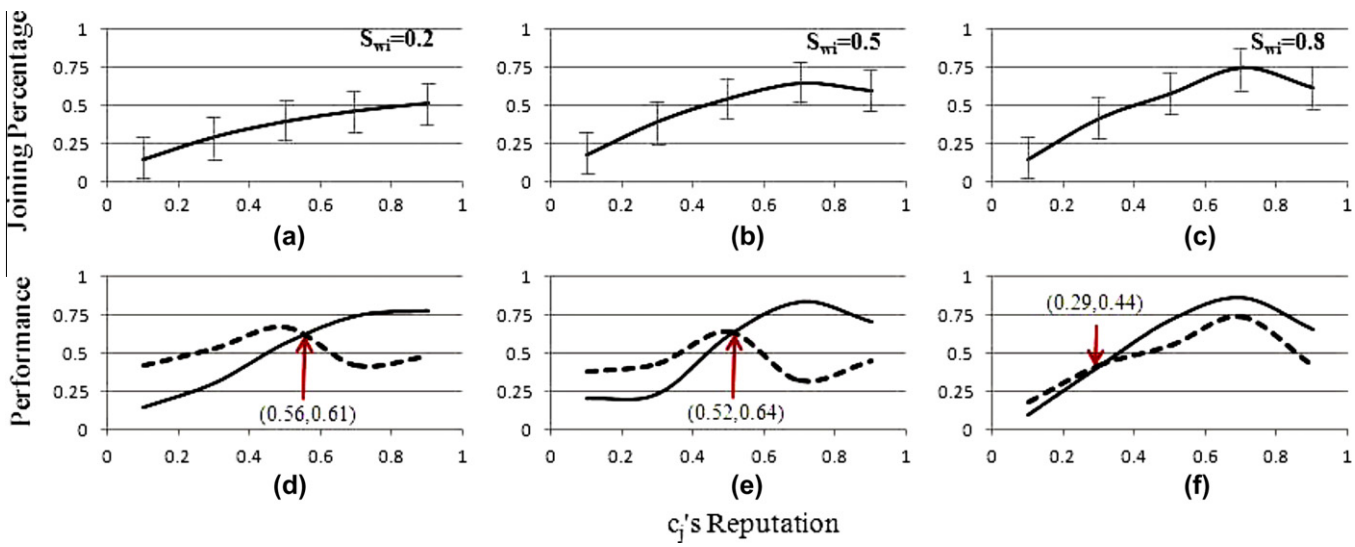

Fig. 2. Performance of three categorized web services on joining a community.

ning steps 300 till 320 (not very stabilized environment). Three typical entities are considered: an individual web service, a joined web service to a community, and a community. Overall, considering our proposed formula to compute the performance, the range of this value for the community is low. Although in general communities attempt to increase their performance, this relatively low value is due to high overall capacity and high number of service requests. We also observe that the highest performance is being held by the web service that is active in a community. Relatively, the corresponding community also holds a reasonable performance. The reputation rate of change reflects the web services' decreasing in reputation (negative values), but the performance is increasing. This is due to the fact that in the observed steps (300–320), the service requests are diminishing and therefore, the web service agent manages to increase its performance. Overall, the gained value is increasing for all these agents, but with diverse rates.

4.2.1. Simulation for joining the community

In order to extract detailed information from the implemented environment and link them to the aforementioned agents' parameters and influential characteristics, we need to focus on detailed empirical analysis over some crucial parameters. We restrict these parameters to performance E_x, risk level S_x, gained value G_x, as well as service request increase and service request decrease. In the rest of this section, we carry on the simulation runs for a specific environment (HFM and GBA) that reflects more realistic settings. We refer to these parameters and confirm our results obtained in Propositions 1, 2, 4 and Corollaries 1 and 2. To this end, we start by investigating the characteristics of the single web service agents that act alone outside the community. During the simulation runs, we set up a number of one-stage games analyzing the strategies that web service agents take in different situations. We repeat the same process using three different classes of the web services according to their risk attributes. Fig. 2 illustrates six plots categorizing three different types of single web service agents (categorized based on their risk levels S_x) that are involved in the one-stage game regarding joining the community. In the plots (a), (b), and (c) the x-axis denotes community's public reputation that is broadcasted by the player 2 (c_j) in the game. The y-axis denotes the percentage of the web service agents that considered joining the community. All the obtained data are based on our observations from the environment in different reputation values regarding the same community. In this experiment, the community is willing to accept joining web services since its market share is not balanced with its limited capacity because the number of received requests is beyond the community's capacity C_{c_j}. As shown in the plots, there are different joining percentages regarding the situation that either encourages or discourages most of the web services. In the same figure (Fig. 2), the plots (d), (e), and (f) illustrate the average performance comparison between the case where a typical analyzed web service agent was acting alone (the dotted curve) and the case where the web service joined the community (the continue curve). The updates in performance clarify the extent to which the joining strategy is chosen wisely. In this experiment, the community adopts its strategies according to its individual performance analysis regardless of the threshold that could lead the web services to join.

We also lunched experiments with the community's representative that is capable of analyzing the threshold that would enhance the control of the master web service agent over the adopting strategies of single web service agents that are willing to join and obtain higher performance. Fig. 3 plots the same group of web services (categorized in plots (a), (b), and (c)) facing a community whose the master web service analyzes the threshold (bold curves) that could encourage the web service agents to join. For comparison reasons, the same experiments are launched where the master web service does not take the

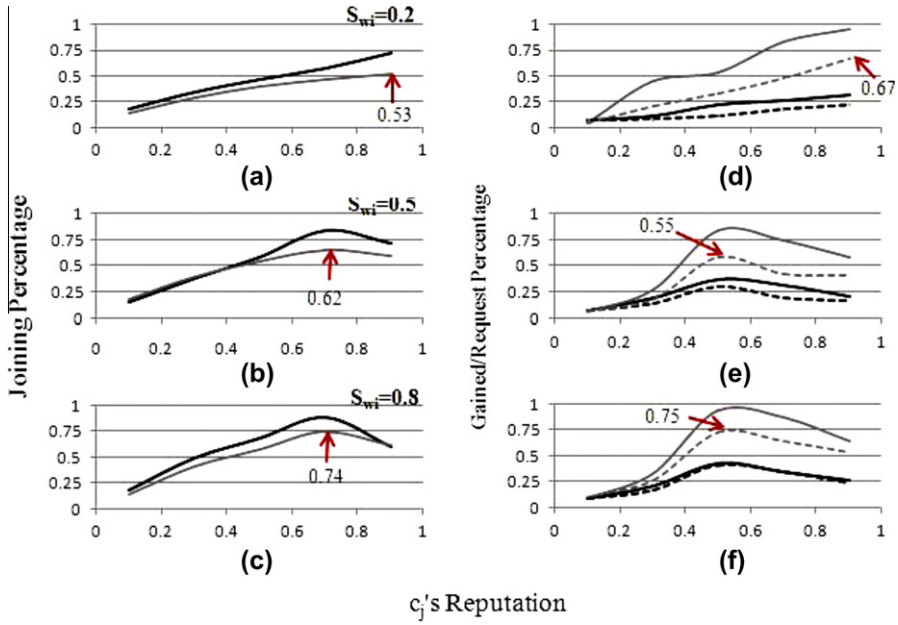

Fig. 3. The impact of threshold on the joining percentage and performance of three categorized web services.

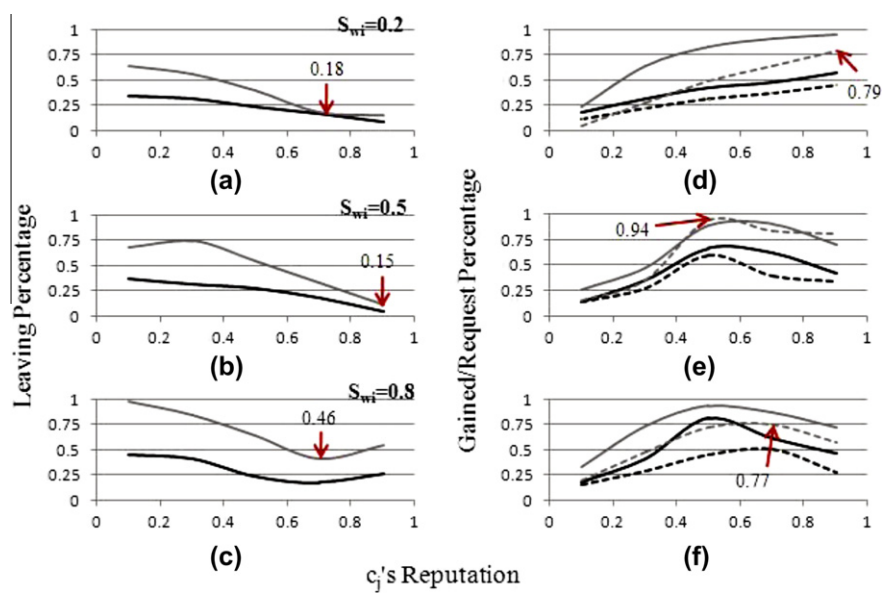

Fig. 4. The impact of threshold on the leaving percentage and performance of three categorized web services.

threshold into account (gray curves). As shown in this figure, c_j is more successful in games with players that hold relatively high risk attribute and the overall efficiency when the threshold is considered is higher. For lower risky web services, the community is more successful in absorbing the web services by advertising higher reputation. This fact is promising according to web services' desire to increase self performance. However, the community facilitates the joining process and meanwhile obtains the control on the strategies that the web service agents adopt. In the same figure (Fig. 3), plots (d), (e), and (f) represent two extra parameters regarding the same experiments: the gained value (G_x) (bold curves) and percentage of request increase/decrease (gray curves). The continue bold and gray curves are those associated with the case where the threshold is investigated and the dotted ones represent the results when such a threshold is not considered. These experiments reflect the extent to which the total gained value and service requests are impacted by the joining percentage to the community.

4.2.2. Simulation for leaving the community

In the previous experiments, we analyzed the case where individual web service agents considered joining a community so that they can act on group. In the following, we analyze experiments we conducted on the performance updates regarding web services that are already members of a community and which are involved in one-stage game facing the community's representative. Fig. 4 illustrates six plots categorizing three types of web services according to their risk level S_{w_i}. These plots illustrate the percentage of leaving the community together with the corresponding gained value and request percentage. As indicated in plots (a),

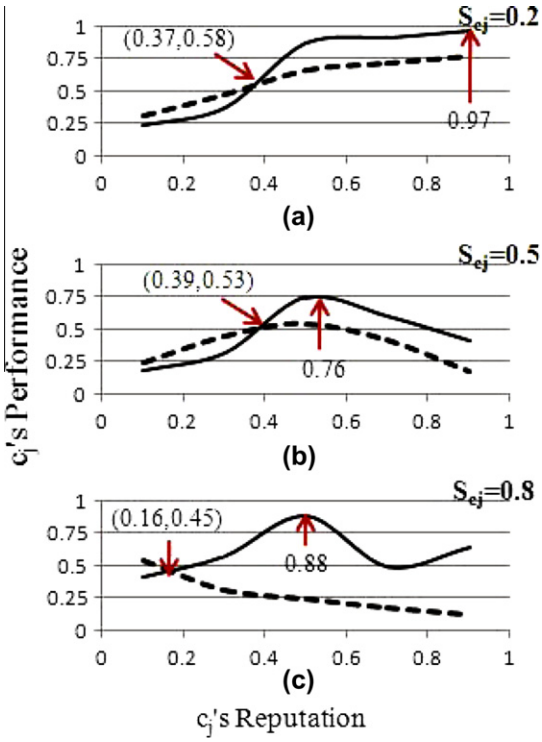

Fig. 5. Performance of three categorized communities of web services.

(b), and (c), the web services act more or less according to their satisfaction of joining the community. Therefore, the percentage of leaving decreases with the increase of the community's reputation. We also observe that when the threshold μ_{w_i} is being analyzed (bold curves), the percentage of leaving is less compared to the case where the threshold is being ignored (gray curves). In plots (d), (e), and (f), the experiment also shows that in general, the gained value (bold curves) and request percentage (gray curves) behave better when the threshold is being studied (continue curves vs. dotted ones).

In Fig. 5, we compare the total performance of different communities categorized based on their risk levels ($S_{c_j} = 0.2$, 0.5, and 0.8). In these plots, the bold curves represent the performance of the community when the threshold μ_{w_i} is taken into account and the dotted ones represent the community when the threshold is being ignored. Again, the simulation shows that the overall performance of communities are enhanced when they consider the computed threshold.

5. Related work

A wide range of models have been proposed to formalize service selection and task allocation in MAS environments of web services (Rosario et al., 2007; Ruth & Shengru, 2007). These models differ from ours in scope, assumptions and computation of the considered parameters. For example, the approaches addressing service selection procedure concentrate on different parameters including service agreements (Kalepu, Krishnaswamy, & Loke, 2003), trust and promises (Jacyno, Bullock, Luck, & Payne, 2009), service providers' reputation (Khosravifar et al., 2010; Rosario et al., 2007; Ruth & Shengru, 2007), and service directories empowered with knowledge (Fardin, Naser, & Ali, 2012). Efforts to handle the optimum service selection led to approximation methods that attempt to balance the parameters associated with the web service as an intelligent service provider to maximize its outcome in short term interactions (Khosravifar et al., 2009, 2010; Maamar et al., 2009; Yang & Cho, 2005). Our proposal concentrates more on maximizing the performance as the key parameter in long-term service selection.

In Ali, Ludwig, and Rana (2005), the proposed framework regulates the service selection based on trust policies expressed by the service consumers. In Maximilien (2005), the authors propose an ontology for quality of service. Users compute the web services' reputation using ratings. The frameworks proposed in Kalepu et al. (2003), Maximilien and Singh (2002) address the reputation issue of web services. In all of these models, the multi-agent network of web services means that active agents represent individual web services. However, the fact that services might collaborate with one another at different stages to improve the offered quality of service has been ignored. To improve these limitations, there are number of proposed models that address coalition formation among agents (Muller, Kowalczyk, & Braun, 2006; Vassileva, Breban, & Horsch, 2002; Yang & Luo, 2007). In general, coalition is known as a group of distinct, autonomous agents that cooperate in order to act as a coherent group in which they can effectively cope with their surrounding environment and thus increase their benefits. In Yang and Luo (2007), a GA-based algorithm is proposed for coalition structure formation which aims at achieving high performance, scalability, and fast convergence rate. In Yang and Cho (2005), an iterated prisoner's dilemma game is used to model the selection dynamics. The authors propose strategic coalition to obtain superior adaptive agents to increase their service quality and performance. In Vassileva et al. (2002), the authors provide micro-level reasoning mechanism for agents to decide for long-term preferences and trust they have in their cooperative partners. The goal is to propose a macro-level mechanism that scale up and test long-term coalition formation outcomes. Their approach minimize communication among the interacting agents and reduce the complexity of coalition formation scenarios. In the same line of research, there have been few models proposing the communities of web services. The objective is to facilitate and improve the process of web service selection and effectively regulate the process of request and task allocation (Jacyno et al., 2009). In Khosravifar et al. (2010), the authors propose a reputation-based architecture for communities and investigate the collusion scenarios that might falsely increase communities' reputation in the network. In Khosravifar et al. (2009), the authors mainly address the overall assessed reputation that is used as a main reason for service selection. In all these models, the proposed solutions do not consider the overall performance as an aggregation of relevant parameters that impact the service selection for a long-term vision.

In general, the recent aforementioned proposals motivate the existence of communities rather than single functional web services, but fail to systematically provide potential benefits and technically compare different scenarios that increase service providers' efficiency. Next to potential benefits of agents working in group, it is important to pay attention to the emerging concepts such as collaboration agreements that should be established, and competitions among the members of the same group. To this end, there is a requirement to investigate the reasons of desiring to work in group or acting individually. Considering all the aforementioned proposed frameworks, we studied the key issues that encourage the rational web service agents to seek for collaborating with others. This helps agents in their decision making to accept or refuse being members of a given community. Overall, this reasoning mechanism enhances the quality of group interactions in the sense that rational interacting agents explore their best strategies, which results in better outcomes.

6. Conclusion

This paper proposes a game-theoretic based model to analyze the best performance characteristics for active services in open networks. The proposed framework measures the performance of the web service agents considering a number of involved factors. The proposed non-zero-sum games measure the threshold that lead to a control of strategies adopted by the single web service.

Our model has the advantage of being simple and taking into account four important factors as shown by the simulation results: (1) rational services seek better performance in the environment; (2) the collaboration concept is well defined if the maximum performance is posed as the main goal; (3) rational web services might meet higher performance either by joining a community (for the sake of collaboration) or acting alone (for managing the task alone); and (4) the community is capable of managing the number of involving web services. As confirmed by the conducted simulations, the resulting model shows that the performance of the community is increasing once the game-theoretic analysis is considered to impose parameters to control the cardinality of the community set. As future work, we plan to consider the user role in the game to obtain more accurate results when users act rationally. Moreover, we plan to investigate the collusion resistant issue within communities. Finally, extending our one-stage games to repeated games with learning techniques is another plan for further research.

References

Ali, A. S., Ludwig, S. A., & Rana, O. F. (2005). A cognitive trust-based approach for web service discovery and selection. In *Proceedings of the 3rd European conference on web services* (pp. 38–40).

Bentahar, J., Khosravifar, B., Serhani, M. A., & Alishahi, M. (2012). On the analysis of reputation for agent-based web services. *Expert Systems with Applications, 39*(16), 12438–12450.

Chalkiadakis, G., & Boutilier, C. (2012). Sequentially optimal repeated coalition formation under uncertainty. *Autonomous Agents and Multi-Agent Systems, 24*(3), 441–484.

Charif, Y., & Sabouret, N. (2011). Dynamic service composition enabled by introspective agent coordination. *Autonomous Agents and Multi-Agent Systems*. http://dx.doi.org/10.1007/s10458-011-9182-5.

Fardin, A. M., Naser, N. B., & Ali, N. M. (2012). Empower service directories with knowledge. *Knowledge-Based Systems, 30*, 172–184.

García-Sáncheza, F., Valencia-Garcíaa, R., Martínez-Béjarb, R., & Fernández-Breis, J. T. (2009). An ontology, intelligent agent-based framework for the provision of semantic webservices. *Expert Systems with Applications, 36*(2), 3167–3187.

Jacyno, M., Bullock, S., Luck, M., Payne, & T. R. (2009). Emergent service provisioning and demand estimation through self-organizing agent communities. In *8th International joint conference on autonomous agents and multiagent systems* (pp. 481–488).

Jurca, R., Faltings, B., & Binder, W. (2007). Reliable QoS monitoring based on client feedback. In *Proceedings of the 16th international world wide web conference* (pp. 1003–1011).

Kalepu, S., Krishnaswamy, S., & Loke, S. W. (2003). A QoS metric for selecting web services and providers. In *Proceedings of the 4th international conference on web information systems engineering workshops* (pp. 131–139).

Khosravifar, B., Bentahar, J., Moazin, A., & Thiran, Ph. (2010). Analyzing communities of web services using incentives. *International Journal of Web Services Research, 7*(3), 30–51.

Khosravifar, B., Bentahar, J., Thiran, P., Moazin, A., & Guiot, A. (2009). An approach to incentive-based reputation for communities of Web services. In *Proceedings of IEEE 7th international conference on web services* (pp. 303–310).

Khosravifar, B., Bentahar, J., Moazin, A., Maamar, Z., & Thiran, P. (2010). Analyzing communities vs. single agent-based web services: Trust perspectives. In *Proceedings of the IEEE international conference on services computing* (pp. 194–201).

Kun, Z., Manwu, X., Hong, Z., & Jian, X. (2010). Agent service matchmaking algorithm for autonomic element with semantic and QoS constraints. *Knowledge-Based Systems, 23*(2), 132–143.

Lomuscio, A., Qu, H., & Solanki, M. (2012). Towards verifying contract regulated service composition. *Autonomous Agents and Multi-Agent Systems, 24*(3), 345–373.

Maamar, Z., Subramanian, S., Thiran, P., Benslimane, D., & Bentahar, J. (2009). An approach to engineer communities of web services: concepts, architecture, operation, and deployment. *International Journal of E-Business Research, 5*(4), 1–21.

Maximilien, E. M. (2005). Multiagent system for dynamic web services selection. In *The 1st workshop on service-oriented computing and agent-based engineering* (pp. 25–29).

Maximilien, E. M., & Singh, M. P. (2002). Conceptual model of Web service reputation. *SIGMOD Record, 31*(4), 36–41.

Muller, I., Kowalczyk, R., & Braun, P. (2006). Towards agent-based coalition formation for service composition. In *Proceedings of of IEEE/WIC/ACM international conference on intelligent agent technology* (pp. 73–80).

Pendharkar, P. C. (2012). Game theoretical applications for multi-agent systems. *Expert Systems with Applications, 39*(1), 273–279.

Rosario, S., Benveniste, A., Haar, S., & Jard, C. (2007). Probabilistic QoS and soft contracts for transaction based web services. In *IEEE international conference on web services* (pp. 126–133).

Ruth, M., & Shengru, T. (2007). Concurrency issues in automating RTS for Web services. In *IEEE international conference on web services* (pp. 1142–1143).

Vassileva, J., Breban, S., & Horsch, M. (2002). Agent reasoning mechanism for long-term coalitions based on decision making and trust. *Computational Intelligence, 18*(4), 583–895.

Vidonia, R., García-Sánchezb, F., Gasparettoa, A., & Martínez-Béjarb, R. (2011). An intelligent framework to manage robotic autonomous agents. *Expert Systems with Application, 38*(6), 7430–7439.

Yahyaoui, H. (2012). A trust-based game theoretical model for Web services collaboration. *Knowledge-Based Systems, 27*, 162–169.

Yang, S. R., & Cho, S. B. (2005). Co-evolutionary learning with strategic coalition for multiagents. *Applied Soft Computing, 5*(2), 193–203.

Yang, J., & Luo, Z. (2007). Coalition formation mechanism in multi-agent systems based on genetic algorithms. *Applied Soft Computing, 7*(2), 561–568.

Yeung, W. L. (2011). A formal and visual modeling approach to choreography based web services composition and conformance verification. *Expert Systems with Applications, 38*(10), 12772–12785.

Yu, X., & Xu, Z. (2012). Directed graph-based multi-agent coalitional decision making. *Knowledge-Based Systems, 35*, 271–278.